

Prototyp „CapSwap“ – Skizze

Beschreibung

CapSwap ist eine B2B-Plattform, die es Unternehmen ermöglicht, Ressourcen direkt miteinander zu tauschen (Bartering). Besteht zwischen zwei Unternehmen Interesse an einem Tausch, können die Details unkompliziert über eine integrierte Chatfunktion geklärt werden. Neu eingestellte Angebote werden automatisch mit bestehenden Interessen abgeglichen; die betroffenen Unternehmen werden benachrichtigt, sobald ein passender Treffer identifiziert wurde.

Funktionale Anforderungen

1. Ein Unternehmen kann eine Rezension zu dem Angebot eines anderen Unternehmens hinterlassen unter der Bedingung, dass deren Anfrage zu dem Angebot akzeptiert wurde und das Angebot von dem anderen Unternehmen als erledigt markiert wurde.
2. Zwei Unternehmen können über einen Textchat miteinander kommunizieren (Chatfunktion).
3. Ein Unternehmen kann passende Angebote anhand der Leistungskategorie filtern (Suchfunktion).
4. Ein Unternehmen kann passende Angebote über eine Textsuche auffinden.
5. Ein Unternehmen kann ein Angebot mit Fotos (zwischen 1 und 10 Fotos), Beschreibung und einer entsprechenden Kategorie erstellen.
6. Ein Unternehmen kann bei ihren eigenen Angeboten Fotos, Beschreibungen und Kategorien nach der Erstellung bearbeiten.
7. Ein Unternehmen kann eine Kategorie abonnieren.
8. Ein Unternehmen kann Benachrichtigungen über relevante Angebote in abonnierten Kategorien erhalten (Benachrichtigungsfunktion).
9. Ein Unternehmen kann Angebote nach Datum sortieren.
10. Ein Unternehmen kann ein eigenes Angebot löschen.
11. Das System löscht Angebote automatisch nach Ablauf von fünf Jahren.
12. Das System informiert den Nutzer im Vorfeld über die anstehende Löschung eines Angebots.
13. Ein Unternehmen kann zu der Anfrage eines anderen Unternehmens ein Angebot mit max. 3 Fotos, einer Nachricht machen.
14. Ein Unternehmen kann ein Angebot annehmen oder ablehnen.
15. Ein Unternehmen kann ein Angebot zurückziehen, soweit diese noch nicht angenommen oder abgelehnt wurde.
16. Ein Unternehmen kann auf der Startseite einloggen oder sich mit einer E-Mail-Adresse und Passwort registrieren.
17. Ein Unternehmen kann das Passwort ändern, indem es die Zusendung eines entsprechenden Links auf die registrierte E-Mail-Adresse beantragt.

Nicht-funktionale Anforderungen

1. Der Server muss bei einer Anfragerate von weniger als 50 Anfragen pro Sekunde jede Anfrage innerhalb von 0,2 Sekunden beantworten.
2. Der Chat muss neue Nachrichten innerhalb von 0,2 Sekunden anzeigen.

3. Das Server-Programm muss auf mehrkernigen Systemen in der Lage sein, die verfügbare Mehrkernigkeit beim Multithreading dynamisch und in Abhängigkeit von der Anzahl der Kerne auszunutzen.
4. Benachrichtigungen über neuen, zu abonnierten Kategorien passende Angebote müssen innerhalb von 1 Minute versendet werden.
5. Die E-Mail für die Passwortänderung muss innerhalb von 5 Minuten geliefert werden
6. Fehlermeldungen (z. B. fehlgeschlagener Login) müssen dem Nutzer innerhalb von 1 Sekunde verständlich gemeldet werden.
7. Server beantwortet korrekte Anfragen zu >99,9% ohne internen Fehler
8. Der Server ist innerhalb von 15s nach Start bereit, Anfragen entgegenzunehmen, wobei das Server-Programm bereits mit allen Dependencies gebaut wurde.
9. Die Datenbank beantwortet die Anfragen innerhalb von 2 Sekunden
10. Alle Nutzeranfragen an die Datenbank werden über Prepared Statement validiert und gebunden, um SQL Injections zu vermeiden
11. Die Datenbanktabellen erfüllen logische Konsistenzregeln, sind vollständig und korrekt.
12. Es können nicht mehrere Nutzer mit der gleichen E-Mail-Adresse erzeugt werden.
13. Bei der Registrierung kann nur eine gültige E-Mail-Adresse verwendet werden

Verteilung

Nebenläufigkeit:

- mehrere Anwender greifen auf die App gleichzeitig zu
- mehrere Anfragen an die Datenbank werden parallel bearbeitet
- Parallele Verarbeitung von REST-API (wie z.B. Angeboterstellung und Suchfunktion oder Favoritenanzeigenspeicherung, während die nächste Seite der Suche geladen wird)

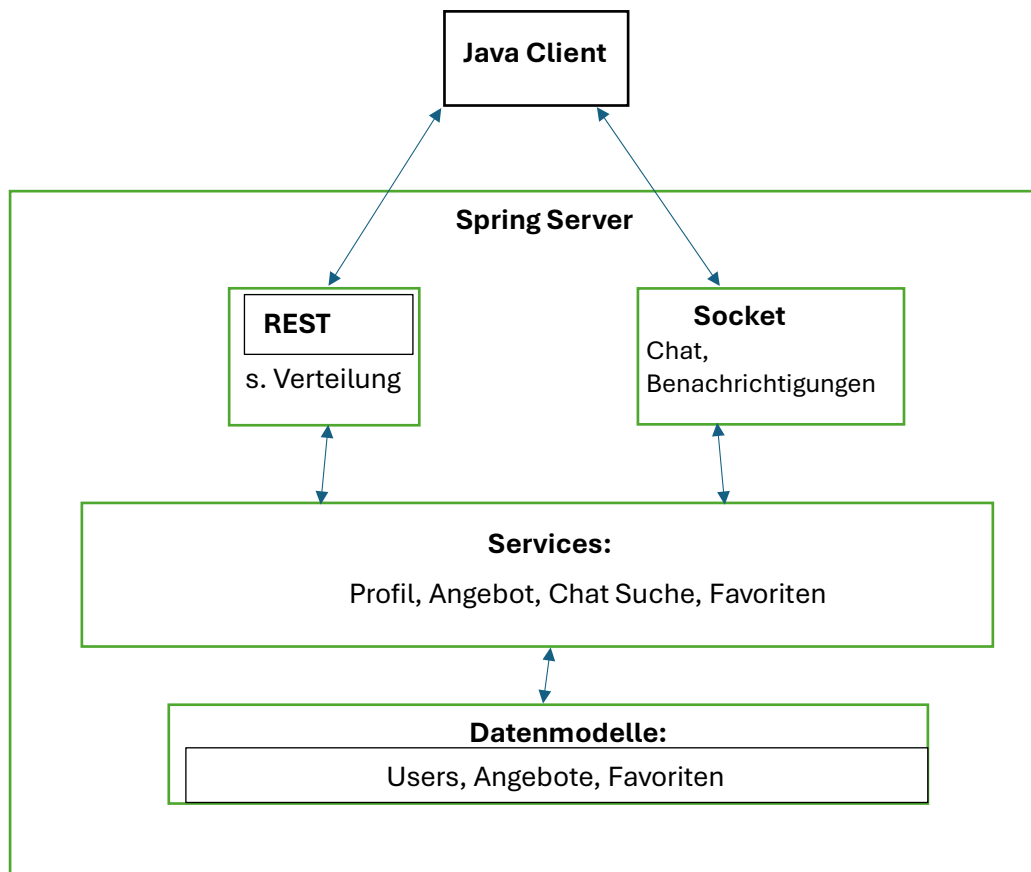
Sockets:

- Benachrichtigungen werden bei passenden Treffern via Sockets an die betroffenen Unternehmen übermittelt.
- Die Chatfunktion zwischen zwei Unternehmen wird über Sockets realisiert.

Komponenten:

- Client-Server Architektur mit Spring Boot
- REST-API-Controller: Suche, Erstellung/Bearbeitung/ Löschung der Angebote, Verwaltung des Profils, Login/Registrierung, Speicherung/ Abonnieung/Bearbeitung der Favoriten
- PostgreSQL-Datenbank: Users, Angebote, Favoriten

Skizze:



Lastsimulation und Test der nicht-funktionalen Anforderungen

Nicht-Funktionale Anforderung	Überprüfung
Das Server-Programm muss auf mehrkernigen Systemen die Mehrkernigkeit dynamisch, abhängig von der Kernanzahl, beim Multithreading ausnutzen.	Multithreaded Tasks (Timer, parallele REST-Verarbeitung) werden geloggt. Auswertung prüft, ob die Last gleichmäßig auf plausible Anzahl an Threads verteilt ist.
Die E-Mail für die Passwortänderung muss innerhalb von 5 Minuten geliefert werden	Automatisierter Test mit lokalem SMTP-Mock misst die Dauer vom Absenden des REST-Requests bis zum Eintreffen der E-Mail im Test-Posteingang (< 5 Min).
Der Chat muss neue Nachrichten innerhalb von 0,2 Sekunden anzeigen.	Integrationstest mit Vanilla Java Socket und CountdownLatch. Zeitmessung zwischen Absenden (outputStream.write()) und Empfang (inputStream.read()) muss < 0,2s betragen.
Mehrere gleichzeitige WebSocketVerbindungen werden stabil verarbeitet	Testclient baut 10 gleichzeitige WebSocket-Verbindungen auf und hält sie offen. Server-Logs werden auf Fehler, Connection-Drops oder Speicherlecks überwacht. Ergebnis: Anzahl stabiler Verbindungen vs. Fehlerrate.
Benachrichtigungen über neuen, zu abonnierten Kategorien passende Angebote müssen innerhalb von 1 Minute versendet werden.	Messung der Zeitspanne zwischen dem Erscheinen des neuen Angebots und dem Versand der zugehörigen Benachrichtigung.
Server beantwortet korrekte Anfragen zu >99,5 % ohne internen Fehler	Anfragenergebnisse werden geloggt. Server-Logs unterscheiden zwischen Client-Fehlern (Fehlercode: 4xx) und internen Fehlern (Fehlercode: 5xx). Testclient sendet ein breites Spektrum gültiger Anfragen. das Verhältnis interner Fehler zu Gesamtanfragen wird ausgewertet.

Der Server muss bei weniger als 50 Anfragen pro Sekunde jede Anfrage in unter 0,2 Sekunden beantworten.	Client sendet Anfragen in angegebener Rate mittels Taktung und misst für jede Anfrage die Antwortzeit. Ergebnis ist die Zahl der Anfragen mit (un-)zufriedenstellender Antwortzeit.
Fehlermeldungen (z. B. fehlgeschlagener Login) müssen dem Nutzer innerhalb von 1 Sekunde verständlich gemeldet werden.	REST-Client sendet bewusst ungültige Logindaten; der Test stoppt die Antwortzeit (< 1s) und validiert das Vorhandensein eines verständlichen Fehlertexts im Response-Body.
Der Server ist innerhalb von 15s nach Start bereit, Anfragen entgegenzunehmen, wobei das Server-Programm bereits mit allen Dependencies gebaut wurde.	Überprüfung in Logs, welche Spring selbstständig anlegt. Dort ist auch Startzeit vermerkt.
Die Datenbank beantwortet die Anfragen innerhalb von 2 Sekunden	Auswertung des aktivierten PostgreSQL Slow-Query-Logs (Schwellenwert auf 2s gesetzt) während eines simulierten Lasttests mit komplexen Suchanfragen.
Alle Nutzeranfragen an die Datenbank werden über Prepared Statement validiert und gebunden, um SQL Injections zu vermeiden	Automatisierter Security-Test mit gängigen SQL-Injection-Strings auf alle API-Eingabefelder; Prüfung, dass die Datenbank diese nicht ausführt und es keine Fehlermeldung gibt.

Die Datenbanktabellen erfüllen logische Konsistenzregeln, sind vollständig und korrekt.	Integrationstest (via Testcontainers) versucht gezielt fehlerhafte Referenzen (z. B. Angebot mit nicht-existierender User-ID) zu speichern; DB muss via Exception (Foreign-Key-Verletzung) ablehnen.
Es können nicht mehrere Nutzer mit der gleichen E-Mail-Adresse erzeugt werden.	Test führt zwei aufeinanderfolgende Registrierungs-Requests mit identischer E-Mail-Adresse aus; der zweite Request muss mit einem Unique-Constraint-Fehler fehlschlagen.